API Explorer - Arquitectura y Documentación Técnica

Resumen

API Explorer es un framework web modular diseñado para proporcionar capacidades dinámicas de exploración y prueba de APIs para módulos Python. Construido sobre FastAPI con soporte WebSocket, ofrece una interfaz moderna y responsiva que permite a múltiples módulos registrar sus APIs en tiempo de ejecución, proporcionando formularios de parámetros interactivos, transmisión de eventos en tiempo real y visualización de datos. El sistema implementa un patrón singleton para accesibilidad global manteniendo operaciones thread-safe y una arquitectura escalable.

Arquitectura General

Patrón Arquitectónico Principal

El API Explorer sigue una **arquitectura de servicio basada en singleton** combinada con patrones **observer** y **registry**, organizando la funcionalidad en capas bien definidas:



Componentes Core y Arquitectura

1. API Explorer Core (api_explorer.py)

Propósito: Servicio singleton central que gestiona todo el framework de exploración de APIs.

Componentes:

- `ApiExplorer` : Clase singleton principal que coordina todas las operaciones
- **Registro de Módulos**: Sistema de registro dinámico para módulos externos
- `WebSocketManager` : Manejador de comunicación en tiempo real
- **Gestión de Rutas**: Generación dinámica de rutas FastAPI
- **Difusión de Eventos**: Sistema de distribución de eventos multi-cliente

Características Clave:

- **Patrón Singleton**: Instancia única thread-safe
- **Registro Dinámico**: Registro de módulos y métodos en tiempo de ejecución
- **Operaciones Thread-Safe**: Lock principal para sincronización
- **Gestión de Puertos**: Detección automática de puertos disponibles
- **Ciclo de Vida del Servidor**: Gestión completa de inicio/parada del servidor

```
# Ejemplo de uso básico
from api_explorer import ApiExplorer

explorer = ApiExplorer.get_instance()
explorer.register_module("mi_modulo", "Mi Módulo", "Descripción")
explorer.add_method_group("mi_modulo", "control", "Métodos de Control")
explorer.register_method("mi_modulo", "control", "iniciar", method_info, handler)

# Broadcasting de eventos
explorer.broadcast_event("mi_modulo", {
    "timestamp": time.time(),
    "data": data,
    "event_type": "data_update"
})
```

Detalles de Arquitectura:

- **Thread Safety**: Lock principal (`_lock`) para operaciones críticas
- **Sistema de Eventos**: Difusión de eventos con throttling básico
- **Integración Sistema de Archivos**: Navegación de directorios del lado del servidor
- **Manejo de Errores**: Logging básico y manejo de excepciones

2. Sistema WebSocket

WebSocketManager (websocket_manager.py)

Propósito: Gestor central de conexiones WebSocket con arquitectura de worker pools.

Componentes:

- `WebSocketManager` : Coordinador principal de conexiones y workers
- `WebSocketWorkerPool` : Pool de workers para procesamiento distribuido
- `WorkerStats` : Métricas de rendimiento por worker
- `FastJSONSerializer` : Serialización JSON optimizada

Características:

- **Arquitectura Worker Pool**: Distribución de carga entre múltiples workers
- **Throttling Adaptativo**: Limitación de mensajes basada en frecuencia
- **Load Balancing**: Distribución round-robin entre workers
- **Performance Tracking**: Métricas de procesamiento

```
# Configuración automática de worker pools
{
    "module_id": "data_stream",
    "worker_count": 4,
    "messages_per_second": estimado,
    "avg_processing_time": medido,
    "queue_utilization": calculado
}
```

Serialización JSON (serialization.py)

Propósito: Sistema de serialización JSON optimizado para transmisión de datos.

Características:

- **orjson Integration**: Uso de orjson cuando está disponible para mayor velocidad
- **Template Caching**: Pre-compilación de estructuras JSON frecuentes
- **Fallback Safety**: Degradación grácil a stdlib json si orjson no está disponible
- **Data Sanitization**: Limpieza automática de datos no serializables

3. Módulo Principal (main.py)

Propósito: Punto de entrada y gestión de servidor.

Componentes:

- `run_explorer()` : Función principal para iniciar el API Explorer
- `main()` : Punto de entrada de línea de comandos
- **Configuración del Servidor**: Configuración de host y puerto

Características:

- **Despliegue Flexible**: Soporta modos de servidor bloqueante y no bloqueante
- **Gestión de Configuración**: Configuración de host y puerto
- **Gestión de Importaciones**: Resolución automática de rutas para importaciones de módulos

4. Arquitectura Frontend Modular

JavaScript Modular (`static/js/`)

Propósito: Arquitectura frontend basada en managers con comunicación event-driven.

Componentes Principales:

APIExplorer Coordinator (script.js)

- **Manager Coordination**: Coordinación central de todos los managers
- **Event-Driven Communication**: Sistema de eventos entre managers
- **Module Initialization**: Inicialización secuencial de módulos

DarkModeManager (darkModeManager.js)

- **Theme Management**: Gestión centralizada de temas oscuro/claro
- **Callback System**: Notificación de cambios de tema a otros managers
- **Chart Integration**: Actualización automática de temas de Chart.js
- **Persistence**: Almacenamiento de preferencias de usuario

WebSocketManager (webSocketManager.js)

- **Connection Management**: Gestión de múltiples conexiones WebSocket
- **Event Buffering**: Buffering para eventos de alta frecuencia
- **Performance Optimization**: Limitación de updates para suavidad visual
- **Error Handling**: Manejo de errores y limpieza de conexiones

ChartManager (chartManager.js)

- **Chart Coordination**: Gestión de múltiples instancias Chart.js
- **Batched Updates**: Actualizaciones por lotes para optimización
- **Throttling**: Throttling basado en frecuencia de datos
- **Multi-Channel Support**: Visualización separada o combinada de canales

EventDisplayManager (eventDisplayManager.js)

- **Event Display**: Coordinación de display de eventos en tiempo real
- **Auto-Scrolling**: Scroll automático
- **Data Table Updates**: Actualización incremental de tablas de datos

FileBrowserManager (fileBrowserManager.js)

- **Server Integration**: Integración con navegación de archivos del servidor
- **Modal Management**: Gestión de modal de navegador de archivos
- **Path Resolution**: Resolución de rutas cross-platform

APIMethodsManager (apiMethodsManager.js)

- **Method Execution**: Ejecución de métodos API
- **Parameter Handling**: Gestión de parámetros dinámicos
- **Template Loading**: Carga automática de plantillas JSON

```
// Ejemplo de arquitectura manager
class APIExplorer {
    constructor() {
        this.darkModeManager = new DarkModeManager();
        this.webSocketManager = new WebSocketManager();
        this.chartManager = new ChartManager();
        this.eventDisplayManager = new EventDisplayManager();
    }

    setupManagerCommunication() {
        // Event-driven communication between managers
        this.darkModeManager.onModeChange((isDark) => {
            this.chartManager.updateTheme(isDark);
        });

        this.webSocketManager.onDataReceived((moduleId, data) => {
            this.chartManager.updateData(moduleId, data);
            this.eventDisplayManager.addDataEvent(moduleId, data);
        });
    }
}
```

Plantilla HTML Responsiva (index.html)

Propósito: Interfaz web dinámica y responsiva con framework Bootstrap.

Componentes Clave:

- **Interfaz de Registro de Módulos**: Generación dinámica de lista de módulos
- **Pestañas de Explorador API**: Interfaz de exploración API por módulo
- **Paneles de Eventos en Tiempo Real**: Visualización de eventos WebSocket
- **Formularios de Parámetros**: Generación dinámica de formularios
- **Modal de Navegador de Archivos**: Navegación del sistema de archivos
- **Visualización de Datos**: Integración Chart.js

Características:

- **Diseño Responsivo**: Layout responsivo basado en Bootstrap
- **Modo Oscuro/Claro**: Sistema de temas con persistencia
- **Contenido Dinámico**: Plantillas Jinja2
- **Actualizaciones en Tiempo Real**: Updates en vivo via WebSocket
- **Formularios Interactivos**: Controles específicos por tipo de parámetro

Estilos CSS (styles.css)

Propósito: Sistema de estilos con soporte de tema dual.

Componentes:

- **Estilos Modo Oscuro**: Implementación completa de tema oscuro
- **Estilos Modo Claro**: Tema claro por defecto
- **Estilos de Componentes**: Estilos consistentes para todos los componentes UI
- **Layout Responsivo**: Principios de diseño responsivo
- **Integración de Gráficos**: Temas Chart.js integrados

Estructuras de Datos y Modelos

1. Información de Módulos

Propósito: Almacenamiento de metadatos de módulos como diccionarios anidados.

Estructura:

```
{
    "module_id": {
        "name": str,
        "description": str,
        "show_data_table": bool,
        "method_groups": {
            "group_id": {
                "name": str,
                "methods": {
                    "method_id": {
                        "name": str,
                        "description": str,
                        "type": str,  # "GET" or "POST"
                        "params": {...}
                    }
                }
            }
        },
        "router": APIRouter
    }
}
```

2. WorkerStats

Propósito: Métricas de rendimiento para worker pools.

Atributos:

- **Message Processing**: Conteo de mensajes procesados por worker
- **Timing Metrics**: Tiempo promedio de procesamiento
- **Queue Utilization**: Utilización de colas de mensajes
- **Activity Tracking**: Timestamp de última actividad

3. Datos de Eventos

Propósito: Estructura básica para eventos WebSocket.

Estructura típica:

```
{
    "timestamp": float,
    "event_type": str,
    "module_id": str,
    "data": dict,
    "visualization_data": dict  # opcional
}
```

Características y Capacidades

1. Visualización de Datos

Integración de Gráficos:

- **Framework Chart.js**: Gráficos profesionales
- **Streaming en Tiempo Real**: Actualizaciones de datos en vivo
- **Soporte Multi-canal**: Visualización separada para diferentes canales de datos
- **Batched Updates**: Actualizaciones por lotes para suavidad visual

Modos de Visualización:

- **Gráficos Combinados**: Todos los canales en un solo gráfico
- **Gráficos Separados**: Gráficos individuales por canal
- **Filtrado por Tipo**: Filtrar display por tipo de datos (EEG, EMG, etc.)
- **Auto-escalado**: Escalado dinámico basado en datos

2. Sistema de Eventos WebSocket

Comunicación en Tiempo Real:

- **Worker Pool Architecture**: Procesamiento distribuido
- **Message Throttling**: Limitación basada en frecuencia
- **Load Balancing**: Distribución entre workers
- **Performance Monitoring**: Métricas de rendimiento

Tipos de Eventos:

- **Data Events**: Eventos de datos con visualización
- **System Events**: Eventos del sistema
- **Important Events**: Eventos prioritarios que no se descartan

3. Serialización JSON

Optimizaciones:

- **orjson Integration**: Uso de orjson cuando está disponible
- **Template Pre-compilation**: Plantillas precompiladas
- **Fallback Safety**: Degradación grácil a stdlib json
- **Data Sanitization**: Limpieza automática de datos

4. Formularios de Parámetros Dinámicos

Tipos de Parámetros Soportados:

- **Texto**: Inputs de texto básicos
- **Números**: Inputs numéricos con validación
- **Rangos**: Sliders con formateo personalizable
- **Selección**: Dropdowns con opciones predefinidas
- **Archivos**: Navegador de archivos integrado
- **JSON Templates**: Áreas de texto con carga automática de plantillas

Patrones de Integración y Uso

1. Patrón de Registro de Módulos

```
# Registro básico de módulo
explorer = ApiExplorer.get_instance()
explorer.register_module("mi_modulo", "Mi Módulo", "Descripción")
explorer.add_method_group("mi_modulo", "control", "Control")
explorer.register_method("mi_modulo", "control", "start", method_info, handler)
```

2. Patrón de Difusión de Eventos

```
# Broadcasting de eventos
event_data = {
    "timestamp": time.time(),
    "event_type": "data_update",
    "data": data,
    "visualization_data": {
        "list_view": formatted_data
    }
}

explorer.broadcast_event("mi_modulo", event_data)
```

3. Patrón de Integración del Servidor

```
# Inicialización del servidor
def setup_api():
    explorer = ApiExplorer.get_instance()

    # Registrar módulos
    register_modules(explorer)

    # Iniciar servidor
    explorer.start_server(host="localhost", port=8000, block=False)
    return explorer
```

Patrones Arquitectónicos Implementados

1. Patrón Singleton

- `ApiExplorer` : Singleton thread-safe
- **Gestión de Estado Global**: Estado consistente
- **Resource Sharing**: Compartición eficiente de recursos

2. Patrón Worker Pool

- **Distributed Processing**: Procesamiento distribuido para WebSockets
- **Load Balancing**: Balanceo de carga automático
- **Adaptive Scaling**: Escalado basado en tipo de módulo

3. Patrón Observer

- **Event Broadcasting**: Difusión de eventos a múltiples clientes
- **Callback System**: Sistema de callbacks para notificaciones

4. Patrón Registry

- **Dynamic Registration**: Registro dinámico de módulos y métodos
- **Runtime Configuration**: Configuración en tiempo de ejecución

Características de Rendimiento

1. Manejo de Datos de Alta Frecuencia

- **Worker Pool Processing**: Procesamiento distribuido
- **Adaptive Throttling**: Throttling inteligente
- **Batch Processing**: Procesamiento por lotes

2. Gestión de Conexiones

- **Worker Pool Architecture**: Arquitectura escalable
- **Performance Monitoring**: Monitoreo de rendimiento
- **Automatic Cleanup**: Limpieza automática de recursos

3. Rendimiento de UI

- **Batched DOM Updates**: Actualizaciones de DOM por lotes
- **Throttled Chart Updates**: Updates de gráficos con throttling
- **Incremental Table Updates**: Actualización incremental de tablas

Stack Tecnológico

Backend:

- **FastAPI**: Framework web asíncrono
- **Uvicorn**: Servidor ASGI
- **orjson** (opcional): Serialización JSON rápida
- **asyncio**: Programación asíncrona
- **WebSockets**: Comunicación bidireccional

Frontend:

- **JavaScript Modular**: Arquitectura de managers
- **Chart.js**: Visualización de datos

- **Bootstrap 5**: Framework CSS responsivo
- **WebSocket APIs**: APIs WebSocket nativas

Herramientas:

- **Worker Pools**: Procesamiento distribuido
- **Throttling**: Limitación de recursos
- **Performance Monitoring**: Monitoreo de métricas